

Bài làm SmartBooking

- **Họ tên:** Lê Trung Kiên
- **MSSV:** 23127075

Phần A - Chọn kiến trúc tổng thể

Em chọn kiến trúc chính cho phiên bản đầu tiên của SmartBooking là **Modular Monolith**.

1. Vì sao chọn Modular Monolith?

SmartBooking ở giai đoạn đầu chưa phải là hệ thống quá lớn: công ty có 5 lập trình viên, 1 DevOps bán thời gian, cần ra sản phẩm đầu tiên trong 3 tháng và dự kiến khoảng 10.000 người dùng trong 6 tháng đầu. Vì vậy, nếu chọn microservices ngay từ đầu thì chi phí vận hành, deployment, monitoring, xử lý lỗi phân tán và giao tiếp giữa các service sẽ khá cao so với năng lực đội ngũ hiện tại.

Modular Monolith phù hợp hơn vì hệ thống vẫn được triển khai như **một ứng dụng**, nhưng bên trong được chia thành các module rõ ràng theo năng lực kinh doanh hoặc bounded context, ví dụ: User, Service Catalog, Booking, Payment, Notification, Staff và Report. Cách này giúp phát triển nhanh như monolith, nhưng vẫn giữ được ranh giới module để sau này có thể tách dần thành microservices nếu hệ thống lớn lên.

Nói cách khác, Modular Monolith là lựa chọn cân bằng giữa tốc độ ra mắt sản phẩm và khả năng mở rộng về sau.

2. Vì sao chưa chọn Monolith?

Monolith đơn giản giúp làm nhanh và deploy dễ, nhưng nếu toàn bộ chức năng bị trộn lẫn trong một khối mã nguồn lớn thì sau một thời gian hệ thống dễ khó hiểu, khó kiểm thử và khó thay đổi. SmartBooking có nhiều nhóm chức năng khác nhau như đặt lịch, thanh toán, thông báo, báo cáo và quản lý nhân viên. Nếu không chia module ngay từ đầu, khi công ty mở rộng thêm chi nhánh sau 1 năm thì việc tách chức năng hoặc scale từng phần sẽ khó hơn.

3. Vì sao chưa chọn Microservices?

Microservices có ưu điểm là các dịch vụ nhỏ, tự trị, ghép nối lỏng, có thể triển khai và mở rộng độc lập. Tuy nhiên, kiến trúc này cũng làm tăng độ phức tạp hệ thống: cần API Gateway, service discovery, CI/CD riêng cho từng service, logging tập trung, monitoring, distributed tracing, retry, timeout, circuit breaker và xử lý nhất quán dữ liệu giữa nhiều database.

Với đội 5 lập trình viên và 1 DevOps bán thời gian, chọn microservices ngay từ đầu có thể làm chậm tiến độ 3 tháng và tạo gánh nặng vận hành không cần thiết. Microservices chỉ nên cân nhắc khi hệ thống đã lớn hơn, có tải cao rõ ràng hoặc có module cần triển khai và scale độc lập.

4. Kiến trúc này có phù hợp với đội hiện tại không?

Có. Modular Monolith phù hợp vì:

- Chỉ cần deploy một ứng dụng nên giảm tải cho DevOps.
- Các lập trình viên vẫn có thể chia nhau phụ trách từng module.

- Ranh giới module rõ ràng giúp giảm phụ thuộc nội bộ.
- Có thể dùng event nội bộ hoặc message broker đơn giản cho các xử lý bất đồng bộ như thông báo, báo cáo và ghi log.

5. Nếu hệ thống lớn lên sau 1 năm, phần nào có thể tách trước?

Các phần nên tách trước là:

- **Notification:** gửi email/SMS là generic subdomain, dễ tách riêng và lỗi thông báo không nên làm hỏng luồng đặt lịch.
- **Payment:** có tích hợp cổng thanh toán bên ngoài, yêu cầu bảo mật, retry và xử lý lỗi riêng.
- **Report/Analytics:** thường đọc nhiều, xử lý bất đồng bộ, có thể dùng CQRS để tách read model khỏi luồng đặt lịch chính.
- **Booking:** nếu số lượng đặt lịch tăng mạnh, đây là core subdomain và có thể cần scale riêng.

Phần B - Chia chức năng hệ thống

Phần chức năng	Nhiệm vụ chính	Có nên tách riêng ngay không? Vì sao?
Người dùng	Quản lý tài khoản, thông tin khách hàng, đăng nhập	Chưa cần tách thành service riêng. Nên là một module trong Modular Monolith vì quy mô ban đầu nhỏ, nhưng cần ranh giới dữ liệu rõ để sau này có thể tách Identity/User Service.
Dịch vụ	Quản lý danh sách dịch vụ, giá, thời lượng, chi nhánh cung cấp	Chưa cần tách riêng ngay. Đây là module quan trọng nhưng ban đầu nghiệp vụ chưa quá phức tạp, có thể để chung ứng dụng và truy cập qua interface nội bộ.
Đặt lịch	Tạo booking, kiểm tra lịch trống, giữ chỗ, xác nhận hoặc hủy lịch	Nên là module trung tâm và có ranh giới rõ nhất. Chưa tách thành microservice ngay, nhưng phải thiết kế như core module vì đây là năng lực kinh doanh chính của SmartBooking.
Thanh toán	Tạo giao dịch, nhận kết quả thanh toán, xử lý thành công/thất bại	Ban đầu có thể là module riêng trong cùng ứng dụng. Nếu tích hợp nhiều cổng thanh toán hoặc tải tăng thì nên tách trước vì có nhiều lỗi ngoại vi và yêu cầu bảo mật.
Thông báo	Gửi email/SMS xác nhận, thông báo hủy lịch hoặc thanh toán	Nên tách logic thành module riêng và xử lý bằng event. Chưa cần deploy riêng ngay, nhưng đây là ứng viên tốt để tách thành service vì là generic subdomain.
Nhân viên	Xem lịch, xác nhận lịch, cập nhật trạng thái phục vụ	Chưa cần tách riêng ngay. Có thể là module Staff dùng dữ liệu booking thông qua API/module interface, tránh truy cập trực tiếp dữ liệu module khác tùy tiện.
Báo cáo	Thống kê số lượng booking, doanh thu, hiệu suất theo chi nhánh	Không nên nằm trong luồng xử lý chính. Nên cập nhật bằng event và có read model riêng. Có thể tách thành Report Service khi dữ liệu lớn hoặc nhu cầu phân tích phức tạp.

Phần C - Luồng đặt lịch

Luồng đặt lịch chính:

```

Người dùng chọn dịch vụ
-> chọn ngày giờ
-> hệ thống kiểm tra lịch trống
-> tạo booking tạm
-> chuyển sang thanh toán
-> thanh toán thành công
-> xác nhận booking
-> phát sự kiện BookingConfirmed
-> gửi email/SMS
-> cập nhật báo cáo
-> ghi log/phân tích dữ liệu
  
```

Trong luồng này, các bước ảnh hưởng trực tiếp đến kết quả người dùng nhìn thấy nên xử lý ngay. Các bước phụ trợ, không bắt buộc người dùng phải chờ, nên xử lý bằng sự kiện để giảm coupling và tăng khả năng phục hồi.

Bước xử lý	Nên xử lý ngay hay xử lý bằng sự kiện?	Lý do
Kiểm tra lịch trống	Xử lý ngay	Người dùng cần biết ngay khung giờ còn trống hay không. Nếu xử lý trễ có thể gây đặt trùng lịch.
Tạo booking	Xử lý ngay	Booking là kết quả chính của thao tác đặt lịch. Hệ thống cần tạo booking tạm hoặc booking pending trước khi thanh toán.
Thanh toán	Xử lý ngay kết hợp event	Người dùng cần biết thanh toán thành công hay thất bại. Tuy nhiên kết quả thanh toán có thể phát event như PaymentSucceeded hoặc PaymentFailed cho các bước sau.
Gửi email/SMS	Xử lý bằng sự kiện	Đây là tác vụ phụ trợ. Nếu email/SMS lỗi thì booking không nên bị hủy. Notification module có thể retry sau.
Cập nhật báo cáo	Xử lý bằng sự kiện	Báo cáo không cần cập nhật tức thì trong luồng đặt lịch. Có thể chấp nhận eventual consistency.
Ghi log/phân tích dữ liệu	Xử lý bằng sự kiện	Phân tích dữ liệu không ảnh hưởng trực tiếp đến kết quả đặt lịch. Xử lý bất đồng bộ giúp giảm thời gian phản hồi.

Phần D - Dùng sự kiện như thế nào?

Ở giai đoạn đầu, SmartBooking có thể dùng event nội bộ trong Modular Monolith hoặc một message broker đơn giản nếu cần tách xử lý bất đồng bộ. Các module giao tiếp bằng event để giảm phụ thuộc: module phát event không cần biết chi tiết module nhận.

Sự kiện	Ai phát ra?	Ai nhận?	Nhận để làm gì?
BookingCreated	Booking	Payment, Staff	Payment tạo yêu cầu thanh toán; Staff có thể thấy booking ở trạng thái chờ.
PaymentSucceeded	Payment	Booking	Booking xác nhận lịch và chuyển trạng thái từ Pending sang Confirmed.
PaymentFailed	Payment	Booking, Notification	Booking hủy booking tạm hoặc đánh dấu PaymentFailed; Notification thông báo thanh toán thất bại nếu cần.
BookingConfirmed	Booking	Notification, Report, Staff	Notification gửi email/SMS; Report cập nhật số lượng booking/doanh thu; Staff hiển thị lịch mới.
BookingCancelled	Booking hoặc Staff	Notification, Report, Payment	Notification gửi thông báo hủy; Report cập nhật thống kê; Payment xử lý hoàn tiền nếu cần.
NotificationSent	Notification	Booking hoặc Audit/Log	Ghi nhận thông báo đã gửi thành công để theo dõi trạng thái.

Các event nên có `eventId`, `eventType`, `occurredAt`, `bookingId`, `userId` và dữ liệu cần thiết. Consumer cần xử lý idempotency để tránh tác động sai nếu một event bị gửi hoặc xử lý nhiều lần.

Phần E - Chọn cách điều phối

1. Với SmartBooking giai đoạn đầu, chọn cách nào?

Em chọn **Broker topology** hoặc cách phát sự kiện đơn giản.

Trong cách này, Booking module phát event như `BookingConfirmed`, còn các module khác như Notification, Report và Staff tự subscribe để xử lý. Không có bộ phận trung tâm điều phối toàn bộ workflow.

2. Vì sao?

Giai đoạn đầu của SmartBooking có workflow chưa quá phức tạp. Sau khi booking được xác nhận, các tác vụ như gửi email/SMS, cập nhật báo cáo và ghi log đều là tác vụ phụ trợ, có thể xử lý độc lập. Broker topology phù hợp vì:

- Đơn giản hơn Mediator topology.
- Loose coupling cao hơn vì producer không cần biết consumer.
- Dễ thêm consumer mới, ví dụ Analytics, mà không sửa luồng đặt lịch chính.
- Phù hợp với đội nhỏ và yêu cầu ra sản phẩm trong 3 tháng.

3. Nếu quy trình thanh toán phức tạp hơn, có nên dùng bộ phận điều phối không?

Có. Nếu thanh toán phức tạp hơn, ví dụ cần giữ chỗ, thanh toán, hoàn tiền, áp mã giảm giá, xử lý nhiều cổng thanh toán, timeout và compensation, thì nên dùng **Mediator topology** hoặc một Booking/Payment Orchestrator.

Khi đó mediator có thể quản lý thứ tự xử lý:

Giữ chỗ

- > yêu cầu thanh toán
- > nếu thành công thì xác nhận booking
- > nếu thất bại thì hủy giữ chỗ
- > nếu lỗi không rõ thì lưu trạng thái để kiểm tra lại

Cách này giống Saga orchestration: mỗi bước là một transaction cục bộ, khi thất bại thì có hành động bồi thường như hủy booking tạm hoặc hoàn tiền.

4. Nếu chỉ gửi email và cập nhật báo cáo, có cần bộ phận điều phối không?

Không cần. Gửi email/SMS và cập nhật báo cáo là các xử lý độc lập sau khi booking đã thành công. Dùng mediator cho trường hợp này sẽ làm hệ thống phức tạp không cần thiết. Chỉ cần Booking phát **BookingConfirmed**, Notification và Report tự xử lý là đủ.

Phần F - Phân tích lỗi thực tế

Tình huống lỗi	Hậu quả	Cách xử lý đề xuất
Thanh toán thành công nhưng booking chưa được xác nhận	Người dùng đã mất tiền nhưng không thấy lịch được xác nhận, gây lỗi nghiêm trọng về trải nghiệm và nghiệp vụ.	Payment module phải lưu giao dịch với trạng thái PaymentSucceeded . Sau đó phát event PaymentSucceeded để Booking xử lý xác nhận. Nếu Booking xử lý lỗi, event không được acknowledge và sẽ được retry. Cần có job đối soát định kỳ giữa giao dịch thanh toán thành công và booking chưa confirmed. Nếu quá thời gian vẫn lỗi thì đưa vào danh sách cần nhân viên xử lý.
Gửi email/SMS bị lỗi	Booking đã thành công nhưng người dùng không nhận được thông báo. Tuy nhiên booking không nên bị hủy vì lỗi notification.	Notification nhận BookingConfirmed và retry tối đa 3 lần, ví dụ sau 1 phút, 5 phút, 15 phút. Nếu vẫn lỗi thì lưu trạng thái NotificationFailed và đưa vào dead-letter queue hoặc danh sách lỗi để nhân viên kiểm tra. Cần lưu notificationId và trạng thái để không gửi trùng quá nhiều lần.
Báo cáo chưa cập nhật kịp	Admin có thể thấy số liệu tạm thời chưa đúng, nhưng không ảnh hưởng trực tiếp đến đặt lịch.	Chấp nhận eventual consistency. Report module consume event BookingConfirmed , BookingCancelled , PaymentSucceeded để cập nhật read model. Nếu consumer lỗi thì event vẫn nằm trong queue và xử lý lại sau. Giao diện báo cáo có thể hiển thị thời điểm cập nhật cuối cùng.
Một sự kiện bị xử lý hai lần	Có thể gửi hai SMS, cộng doanh thu hai	Mỗi event cần có eventId . Consumer phải idempotent bằng cách lưu các event đã xử lý hoặc dùng khóa duy nhất theo eventId/bookingId . Ví dụ Report chỉ cộng doanh thu

Tình huống lỗi	Hậu quả	Cách xử lý đề xuất
	lần hoặc tạo dữ liệu báo cáo sai.	một lần cho mỗi PaymentSucceeded , Notification tránh gửi lại nếu notification đã ở trạng thái Sent .
Hệ thống thanh toán phản hồi chậm	Người dùng phải chờ lâu, booking bị treo ở trạng thái không rõ ràng.	Đặt timeout cho request thanh toán. Booking giữ trạng thái PaymentPending thay vì xác nhận ngay. Nếu cổng thanh toán phản hồi sau bằng webhook thì Payment module cập nhật lại và phát PaymentSucceeded hoặc PaymentFailed . Có thể dùng circuit breaker nếu cổng thanh toán lỗi liên tục để tránh làm nghẽn hệ thống.
Số lượng người dùng tăng đột biến	Ứng dụng có thể phản hồi chậm, đặc biệt ở luồng xem dịch vụ, kiểm tra lịch trống và đặt booking.	Tối ưu truy vấn, cache danh sách dịch vụ, scale ngang ứng dụng Modular Monolith nếu có thể, tách các tác vụ phụ trợ sang queue. Nếu tải tiếp tục tăng, tách Booking, Notification hoặc Report thành service riêng theo mức chịu tải thực tế.
Một module bị lỗi làm ảnh hưởng phần khác	Vì vẫn là một ứng dụng deploy chung, lỗi trong một module có thể ảnh hưởng toàn hệ thống.	Tạo ranh giới module rõ ràng, không cho module truy cập dữ liệu tùy tiện. Các tác vụ phụ trợ chạy bất đồng bộ qua event để lỗi Notification hoặc Report không chặn Booking. Cần logging, monitoring và test theo từng module.

Kết luận

Với SmartBooking phiên bản đầu tiên, **Modular Monolith kết hợp Event-Driven Architecture ở mức vừa phải** là lựa chọn phù hợp nhất. Kiến trúc này giúp nhóm nhỏ phát triển nhanh, vận hành đơn giản, nhưng vẫn có ranh giới module rõ để mở rộng sau này. Các bước quan trọng như kiểm tra lịch trống, tạo booking và xác nhận thanh toán nên xử lý ngay. Các bước phụ trợ như gửi email/SMS, cập nhật báo cáo và ghi log nên xử lý bằng event theo Broker topology.

Nếu sau 1 năm hệ thống phát triển mạnh, nhóm có thể tách dần các module có nhu cầu độc lập cao như Notification, Payment, Report và Booking thành microservices. Khi đó cần bổ sung API Gateway, database per service, observability, Saga, retry, idempotency và dead-letter queue để vận hành hệ thống phân tán an toàn hơn.

Đã dùng AI như thế nào?

Em dùng AI để hỗ trợ hiểu và hệ thống hóa các khái niệm trong tài liệu tham khảo như Monolith, Modular Monolith, Microservices, Event-Driven Architecture, Broker topology, Mediator topology, loose coupling, Saga, eventual consistency và idempotency. Em cũng dùng AI để gợi ý cách trình bày lập luận, sau đó tự đọc lại và điều chỉnh nội dung cho phù hợp với tình huống SmartBooking.